

FIRST ASSIGNMENT REPORT

Defining A Cross-Correlation Function on A Gait Phase Signal



Name : Jeremia Christ Immanuel Manalu
NRP : 5023231017
Course : Biomodelling (A)
Class : A
Lecturer : Eko Agus Suprayitno, S.Si., M.T.
Department : Biomedical Engineering

**FACULTY OF INTELLIGENT ELECTRICAL AND INFORMATICS
TECHNOLOGY**

INSTITUT TEKNOLOGI SEPULUH NOPEMBER

2025

CHAPTER I. FUNDAMENTAL THEORY

1.1. Gait Phase

Gait analysis is the systematic study of human locomotion. A single gait cycle is the interval of time from one event of a specific foot to the next occurrence of that same event with the same foot. In clinical and biomechanical settings, gait analysis is a crucial tool for diagnosing neuromuscular disorders, assessing rehabilitation progress, and designing prosthetic limbs. The program analyses gait by using time-series data captured from sensors, specifically Force Sensitive Resistors (FSRs), placed on the Heel and the Toe of a subject. These sensors produce a signal whose amplitude corresponds to the pressure exerted at that point during the walking cycle. The key events captured by these sensors are:

- Heel-Strike is the moment the heel makes initial contact with the ground. This is represented by a rising signal in the Heel data array.
- Toe-Off is the moment the toe pushes off the ground to propel the body forward. This is represented by a peak in the Toe data array just before the signal value drops.

The data from these sensors, Heel[n] and Toe[n], where n is the discrete sample index, are the two fundamental signals used for the subsequent correlation analysis. By examining these signals, we can understand the rhythm, timing, and coordination of a person's walk.

1.2. Autocorrelation

The Autocorrelation Function (ACF) is a mathematical tool used to measure the degree of self-similarity within a single signal as a function of a time delay, or "lag." In essence, it involves comparing a signal with a time-shifted version of itself. In the context of gait analysis, this is incredibly useful for identifying periodicity. Since walking is a repetitive action, the signal from the Toe or Heel sensor is expected to have a repeating pattern. The autocorrelation of the Toe signal, for example, will show a high value (a peak) at a lag that corresponds to the stride time, the duration of one full gait cycle. The first major peak after the initial peak at lag zero indicates the fundamental period of the signal. The program calculates the autocorrelation of the Toe signal (RxxToe) to identify these repetitive characteristics.

$$R_{xx}(k) = \sum_{i=0}^{N-1-k} x[n] * x[n + k]$$

Where:

- $R_{xx}(k)$ is the autocorrelation value at lag k.
- $x[n]$ is the value of the signal at sample index n.
- N is the total number of samples (referred to as Sample in the code).
- k is the time lag (referred to as L in the code).

1.3. Cross-Correlation

The Cross-Correlation Function is a measure of the similarity between two different signals as a function of the time lag applied to one of them. While autocorrelation measures the internal rhythm of a single signal, cross-correlation measures the temporal relationship between two signals. For gait analysis, cross-correlation is the ideal tool for quantifying the coordination between the heel and toe movements. By calculating the cross-correlation of the Heel signal ($x[n]$) and the Toe signal ($y[n]$), we can determine the typical time delay between the heel-strike and the subsequent toe-off. A peak in the cross-correlation function

at a specific lag k indicates that the Heel signal is most similar to the Toe signal when the Toe signal is shifted by k samples. This value of k directly corresponds to the timing difference between these two critical gait events. The program calculates this for both positive and negative lags to determine which event leads the other. For two discrete signals, $x[n]$ (Heel) and $y[n]$ (Toe), each with N samples, the unnormalized cross-correlation R_{xy} at a time lag k is given by:

$$R_{xy}(k) = \sum_n^{N-k-1} x[n] * y[n + k]$$

Where:

- $R_{xy}(k)$ is the cross-correlation value at lag k .
- $x[n]$ is the value of the first signal (Heel) at sample n .
- $y[n]$ is the value of the second signal (Toe) at sample n .
- N and k are the total samples and time lag, respectively.

1.4. Normalization

Raw correlation values are dependent on the amplitude and energy of the signals being analyzed. A signal with a larger amplitude will produce much larger raw correlation values than a signal with a smaller amplitude, even if their underlying patterns are identical. This makes it difficult to compare results or establish a consistent threshold for what constitutes a "strong" correlation. Normalization is the process of scaling the raw correlation output to a standardized range. The specific method used in this program is a biased estimate of the correlation. This method adjusts the raw correlation value by considering how many data points were used in its calculation. At a lag of $k=0$, all N samples of the signals overlap. At a larger lag k , only $N-|k|$ samples overlap. To provide a fair comparison, this normalization method divides the raw correlation sum by the number of overlapping points. This gives an average correlation value per overlapping pair of samples. The normalized cross-correlation, $\rho_{xy}(k)$, is calculated by taking the raw cross-correlation value $R_{xy}(k)$ and dividing it by the number of overlapping samples:

Window technique is used to eliminate spectral leakage effects in frequency domain analysis. In this problem, two types of windows are used:

$$\rho_{xy}(k) = R_{xy}(k) / (N - |k|)$$

Where:

- $\rho_{xy}(k)$ is the normalized cross-correlation value at lag k .
- $R_{xy}(k)$ is the raw cross-correlation value from the formula in section 1.3.
- N is the total number of samples.
- $|k|$ is the absolute value of the lag k .

CHAPTER II. RESULT AND ANALYSIS

2.1. Problems Statement

1. Given: Two pieces of gait pattern data, heel and toe, are in the files Heel123.txt and Toe123.txt.
2. Question: Develop a computer program to determine the cross-correlation function of the above signals. Prepare a report.

2.2. Code Explanation

a. Data Input (via btnInputDataClick)

```
1. procedure TForm1.btnInputDataClick(Sender: TObject);
2. var
3.   heelCount, toeCount: Integer;
4. begin
5.   ButtonClearClick(nil);
6.
7.   OpenFileDialog1.Title := 'Please select the HEEL data file (e.g., Heel123.txt)';
8.   if OpenFileDialog1.Execute then
9.   begin
10.    heelCount := ReadDataFile(OpenFileDialog1.FileName, Heel);
11.   end else
12.   begin
13.    ShowMessage('Heel data loading cancelled.');
```

```
14.    Exit;
15.   end;
16.
17.   OpenFileDialog1.Title := 'Please select the TOE data file (e.g., Toe123.txt)';
18.   if OpenFileDialog1.Execute then
19.   begin
20.    toeCount := ReadDataFile(OpenFileDialog1.FileName, Toe);
21.   end else
22.   begin
23.    ShowMessage('Toe data loading cancelled.');
```

```
24.    Exit;
25.   end;
26.
27.   if (heelCount = 0) or (toeCount = 0) then
28.   begin
29.    ShowMessage('Error reading data or files are empty. Please check the
    files.');
```

```
30.    ButtonClearClick(nil);
31.    Exit;
32.   end;
33.
34.   Sample := Min(heelCount, toeCount);
```

```

35. SetLength(Heel, Sample);
36. SetLength(Toe, Sample);
37.
38. PlotInputData;
39. btnCompute.Enabled := Sample > 0;
40. end;

```

The process begins when the user clicks the "Input Data" button. The btnInputDataClick procedure prompts the user to select two files sequentially: first Heel123.txt and then Toe123.txt. It uses a helper function, ReadDataFile, to read each single-column text file and populate the corresponding Heel and Toe TArray<Double>. To ensure data integrity, the total number of samples (Sample) is set to the minimum length of the two files.

b. Visualization of the Input Signal (plotInputData)

```

1. procedure TForm1.PlotInputData;
2. var
3.   i: Integer;
4. begin
5.   SeriesHeel.Clear;
6.   SeriesToe.Clear;
7.   SeriesToeShifted.Clear;
8.
9.   SeriesHeel.Title := 'Heel (x)';
10.  SeriesToe.Title := 'Toe (y)';
11.  SeriesToeShifted.Title := 'Toe (y shifted)';
12.
13.  for i := 0 to Sample - 1 do
14.  begin
15.    SeriesHeel.AddXY(i, Heel[i]);
16.    SeriesToe.AddXY(i, Toe[i]);
17.  end;
18.
19.  if Sample > 0 then
20.  begin
21.    TrackBarLag.Min := -(Sample - 1);
22.    TrackBarLag.Max := (Sample - 1);
23.  end
24.  else
25.  begin
26.    TrackBarLag.Min := 0;
27.    TrackBarLag.Max := 10;
28.  end;
29.
30.  TrackBarLag.Position := 0;
31.  EditLag.Text := '0';
32.  TrackBarLag.Enabled := True;

```

```

33. EditLag.Enabled := True;
34.
35. UpdateShiftedToeChart(0);
36. end;

```

Once loaded (the data from btnInputDataClick), the PlotInputData procedure is called. This procedure clears any previous data and populates the ChartGait component, plotting the Heel and Toe signals against their sample index number on the x-axis. It also initializes the interactive TTrackBar and TEdit components, setting their range based on the number of samples.

c. *Static Correlation Computations (via btnComputeClick)*

```

1. procedure TForm1.btnComputeClick(Sender: TObject);
2. var
3.   L, n: Integer;
4.   sigmaL: Real;
5.   RxxToe: TArray<Double>;
6.   Rxy: TArray<Double>;
7.   NormRxy: TArray<Double>;
8.   maxLag, lagRange, lagOffset: Integer;
9.   sx, sy, srxy: string;
10. begin
11.   if Sample <= 0 then
12.     begin
13.       ShowMessage('Please input data first. ');
14.       Exit;
15.     end;
16.
17.   // 1. AUTOCORRELATION of Original Toe Data
18.   ...
19.
20.   // 2. CROSS-CORRELATION - Original Heel vs Toe
21.   ...
22.
23.   // 3. NORMALIZATION of CROSS-CORRELATION
24.   ...
25.   CalculateAndUpdateShiftedResults(0);
26.   PageControlResults.ActivePage := TabAutocorr;
27. end;

```

Clicking the "Compute" button triggers the main analysis of the *original, un-shifted* signals. This procedure performs three key calculations as defined in Chapter 1, that are the Autocorrelation, Cross-Correlation, and Normalization. The three functions will be explained in the next section.

d. *Autocorrelation*

```

1. // 1. AUTOCORRELATION of Original Toe Data

```

```

2.  ListBoxAutoCorr.Clear;
3.  SetLength(RxxToe, Sample);
4.  for L := 0 to Sample - 1 do
5.  begin
6.    sigmaL := 0;
7.    for n := 0 to Sample - 1 - L do
8.      sigmaL := sigmaL + Toe[n] * Toe[n + L];
9.    RxxToe[L] := sigmaL;
10.  ListBoxAutoCorr.Items.Add(Format('Rxx[%d] = %8.3f', [L,
    RxxToe[L]]));
11. end;

```

The part of the code above will calculate the autocorrelation of the original Toe signal by implementing the formula:

$$R_{xx}(k) = \sum_{i=0}^{N-1-k} x[n] * x[n + k]$$

The results are displayed in the ListBoxAutoCorr on the "Autocorr" tab.

e. *Cross-Correlation*

```

1. // 2. CROSS-CORRELATION - Original Heel vs Toe
2.  ListBoxCrossCorr.Clear;
3.  SeriesCrossCorr.Clear;
4.  maxLag := Sample - 1;
5.  lagRange := 2 * maxLag + 1;
6.  lagOffset := maxLag;
7.  SetLength(Rxy, lagRange);
8.
9.  for L := -maxLag to maxLag do
10. begin
11.  sigmaL := 0;
12.  if L >= 0 then
13.    for n := 0 to Sample - 1 - L do
14.      sigmaL := sigmaL + Heel[n] * Toe[n + L]
15.  else
16.    for n := 0 to Sample - 1 + L do
17.      sigmaL := sigmaL + Heel[n - L] * Toe[n];
18.  Rxy[L + lagOffset] := sigmaL;
19.  SeriesCrossCorr.AddXY(-L, sigmaL);
20. end;
21.
22. for L := 0 to Sample - 1 do
23. begin
24.  sx := FloatToStrF(Heel[L], ffFixed, 5, 3);
25.  sy := FloatToStrF(Toe[L], ffFixed, 5, 3);
26.  srxy := FloatToStrF(Rxy[L + lagOffset], ffFixed, 8, 3);

```

```

27. ListBoxCrossCorr.Items.Add(Format('x[%d]=%s y[%d]=%s
    rxy[%d]=%s', [L, sx, L, sy, L, srxy]));
28. end;

```

This block of code will compute the cross-correlation between the Heel and Toe signals using the formula:

$$R_{xy}(k) = \sum_n^{N-k-1} x[n] * y[n+k]$$

for all possible lags (-Sample to +Sample). The results are plotted on ChartCrossCorr and listed in ListBoxCrossCorr in the format x[i]=... y[i]=... rxy[i]=....

f. Normalization

```

1. // 3. NORMALIZATION of CROSS-CORRELATION
2. ListBoxNorm.Clear;
3. SeriesNorm.Clear;
4. SetLength(NormRxy, lagRange);
5.
6. ListBoxNorm.Items.Add('Lag' + #9 + 'Normalized Rxy');
7. for L := -maxLag to maxLag do
8. begin
9. // Using the normalization method: Rxy(L) / (N - |L|)
10. // This scales the result based on the number of overlapping samples.
11. if (Sample - Abs(L)) > 0 then
12. begin
13. NormRxy[L + lagOffset] := Rxy[L + lagOffset] / (Sample - Abs(L));
14. end
15. else
16. begin
17. NormRxy[L + lagOffset] := 0;
18. end;
19.
20. ListBoxNorm.Items.Add(Format('%d'#9'%8.3f', [L, NormRxy[L +
    lagOffset]]));
21. SeriesNorm.AddXY(-L, NormRxy[L + lagOffset]);
22. end;

```

Finally, the raw cross-correlation results are normalized using the biased estimate formula:

$$\rho_{xy}(k) = R_{xy}(k) / (N - |k|)$$

These normalized values are then displayed in the ChartNorm and ListBoxNorm on the "Normalization" tab.

g. Time Lag Shifting (via TrackBarLagChange and EditLagKeyPress)

```

1. procedure TForm1.EditLagKeyPress(Sender: TObject; var Key: Char);
2. var
3. val: Integer;

```



```

4. begin
5.   if Key = #13 then
6.     begin
7.       if TryStrToInt(EditLag.Text, val) then
8.         begin
9.           if val < TrackBarLag.Min then val := TrackBarLag.Min;
10.          if val > TrackBarLag.Max then val := TrackBarLag.Max;
11.          TrackBarLag.Position := val;
12.        end
13.      else
14.        begin
15.          EditLag.Text := IntToStr(TrackBarLag.Position);
16.        end;
17.      Key := #0;
18.    end;
19. end;
20.
21. procedure TForm1.TrackBarLagChange(Sender: TObject);
22. var
23.   shiftValue: Integer;
24. begin
25.   shiftValue := TrackBarLag.Position;
26.   EditLag.Text := IntToStr(shiftValue);
27.   if Sample > 0 then
28.     begin
29.       UpdateShiftedToeChart(shiftValue);
30.       CalculateAndUpdateShiftedResults(shiftValue);
31.     end;
32. end;

```

This part of the code is the core interactive feature. When the user either slides the TTrackBar or enters a value into the TEdit box and presses Enter, the TrackBarLagChange event is triggered. This event performs two actions, that are the UpdateShiftedToeChart and CalculateAndUpdateShiftedResults.

h. UpdateShiftedToeChart

```

1. procedure TForm1.UpdateShiftedToeChart(const AShift: Integer);
2. var
3.   i: Integer;
4. begin
5.   SeriesToeShifted.Clear;
6.   for i := 0 to Sample - 1 do
7.     begin
8.       if (i - AShift >= 0) and (i - AShift < Sample) then
9.         begin
10.          SeriesToeShifted.AddXY(i, Toe[i - AShift]);
11.        end;
12.     end;

```

```
13. end;
```

This procedure visually updates the ChartGait by redrawing the SeriesToeShifted line. For a given shift AShift, it plots the point (i, Toe[i - AShift]), creating the effect of the Toe signal moving left or right relative to the Heel signal.

i. *CalculateAndUpdateShiftedResults*

```
1. procedure TForm1.CalculateAndUpdateShiftedResults(const AShift:
   Integer);
2. var
3.   ShiftedToe: TArray<Double>;
4.   i, L, n: Integer;
5.   sigmaL: Real;
6.   RxxShiftedToe: TArray<Double>;
7.   ShiftedRxy: TArray<Double>;
8.   maxLag, lagRange, lagOffset: Integer;
9.   sx, sy_shifted, srxy_shifted: string;
10.  ShiftedNormRxy: TArray<Double>;
11. begin
12.  if (Sample <= 0) or not Assigned(ListBoxShiftedAutoCorr) then Exit;
13.
14.  SetLength(ShiftedToe, Sample);
15.  for i := 0 to Sample - 1 do
16.  begin
17.    if (i - AShift >= 0) and (i - AShift < Sample) then
18.      ShiftedToe[i] := Toe[i - AShift]
19.    else
20.      ShiftedToe[i] := 0;
21.  end;
22.
23.  // 1. Calculate SHIFTED AUTOCORRELATION (for ShiftedToe)
24.  ListBoxShiftedAutoCorr.Clear;
25.  SetLength(RxxShiftedToe, Sample);
26.  ListBoxShiftedAutoCorr.Items.Add('ACF for Toe data shifted by ' +
    IntToStr(AShift));
27.  ListBoxShiftedAutoCorr.Items.Add('Lag' + #9 + 'Rxx');
28.  for L := 0 to Sample - 1 do
29.  begin
30.    sigmaL := 0;
31.    for n := 0 to Sample - 1 - L do
32.      sigmaL := sigmaL + ShiftedToe[n] * ShiftedToe[n + L];
33.    RxxShiftedToe[L] := sigmaL;
34.    ListBoxShiftedAutoCorr.Items.Add(Format('%d  %8.3f', [L,
    RxxShiftedToe[L]]));
35.  end;
36.
```

```

37. // 2. Calculate SHIFTED CROSS-CORRELATION (Heel vs ShiftedToe)
38. ListBoxShiftedCrossCorr.Clear;
39. SeriesShiftedCrossCorr.Clear;
40. maxLag := Sample - 1;
41. lagRange := 2 * maxLag + 1;
42. lagOffset := maxLag;
43. SetLength(ShiftedRxy, lagRange);
44.
45. for L := -maxLag to maxLag do
46. begin
47.   sigmaL := 0;
48.   if L >= 0 then
49.     for n := 0 to Sample - 1 - L do
50.       sigmaL := sigmaL + Heel[n] * ShiftedToe[n + L]
51.     else
52.       for n := 0 to Sample - 1 + L do
53.         sigmaL := sigmaL + Heel[n - L] * ShiftedToe[n];
54.       ShiftedRxy[L + lagOffset] := sigmaL;
55.       SeriesShiftedCrossCorr.AddXY(-L, sigmaL);
56.     end;
57.
58. for L := 0 to Sample - 1 do
59. begin
60.   sx := FloatToStrF(Heel[L], ffFixed, 5, 3);
61.   sy_shifted := FloatToStrF(ShiftedToe[L], ffFixed, 5, 3);
62.   srxy_shifted := FloatToStrF(ShiftedRxy[L + lagOffset], ffFixed, 8, 3);
63.   ListBoxShiftedCrossCorr.Items.Add(Format('x[%d]=%s y_sh[%d]=%s
rxy_sh[%d]=%s',
64.     [L, sx, L, sy_shifted, L, srxy_shifted]));
65. end;
66.
67. // 3. Calculate SHIFTED NORMALIZATION (of Shifted Cross-Corr)
68. ListBoxShiftedNorm.Clear;
69. SeriesShiftedNorm.Clear;
70. SetLength(ShiftedNormRxy, lagRange);
71.
72. ListBoxShiftedNorm.Items.Add('Lag' + #9 + 'Normalized Shifted Rxy');
73. for L := -maxLag to maxLag do
74. begin
75.   // Using the normalization method: Rxy(L) / (N - |L|)
76.   if (Sample - Abs(L)) > 0 then
77.     begin
78.       ShiftedNormRxy[L + lagOffset] := ShiftedRxy[L + lagOffset] / (Sample
- Abs(L));
79.     end
80.   else

```

```

81. begin
82.     ShiftedNormRxy[L + lagOffset] := 0;
83. end;
84.
85. ListBoxShiftedNorm.Items.Add(Format('%d %8.3f', [L,
    ShiftedNormRxy[L + lagOffset]]));
86. SeriesShiftedNorm.AddXY(-L, ShiftedNormRxy[L + lagOffset]);
87. end;
88. end;

```

This procedure mirrors the logic of `btnComputeClick` but operates on a temporary `ShiftedToe` array, which is generated based on the current lag value from the `TTrackBar`. The results are populated into the dedicated "Shifted" tabs: `TabShiftedAutocorr`, `TabShiftedCrossCorr`, and `TabShiftedNormalization`. This provides immediate feedback, allowing the user to see how the correlation peaks and values change as the temporal alignment of the two signals is manually adjusted.

2.3. Result of the Program and Analysis

*Initial GUI

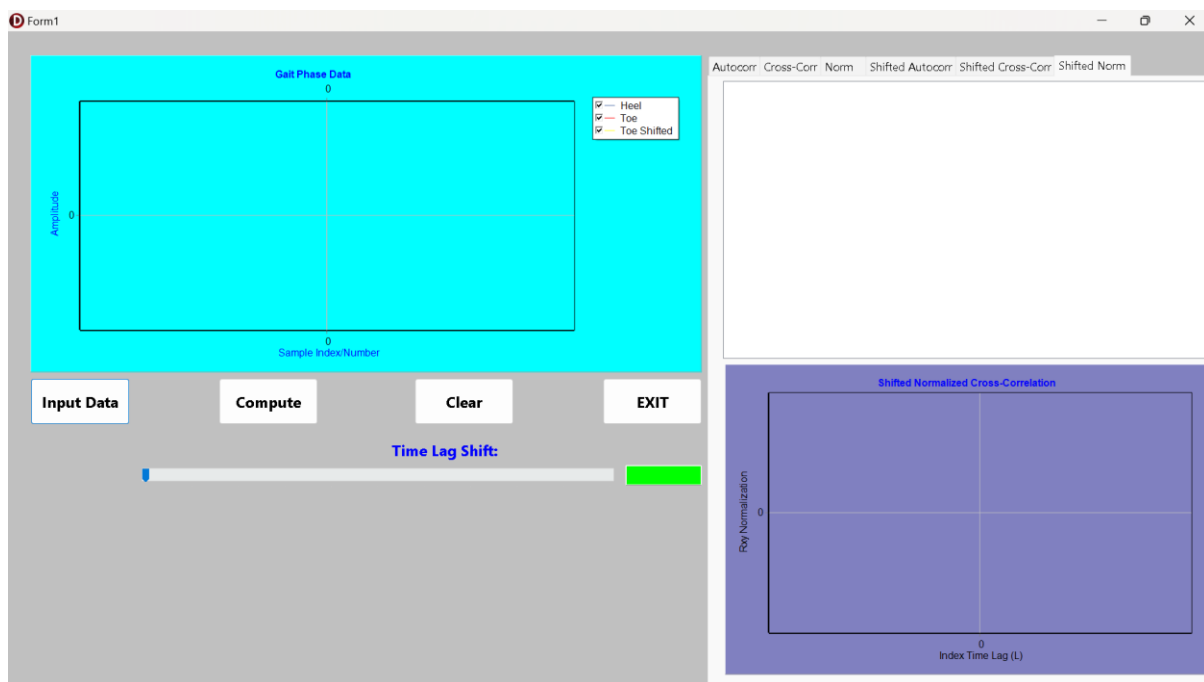


Figure 1. Initial GUI components

This section will explain the results obtained from the gait analysis program. It covers the initial analysis of the original gait data, the effects of interactively shifting the data, a comparison with the target results from the lecturer's program, and an overall evaluation of the tool's performance. The main focus of this section is in the Cross-Correlation and Normalization tab (in `PageControlResults`) and its shifted version, because the Autocorrelation only include the Toe data (both original and shifted).

2.2.1. Input the original Gait Data (Time Lag = 0)

Upon loading the Heel123.txt and Toe123.txt files and clicking the "Compute" button, the program performs an initial, static analysis with a time lag of zero. In this state, the "Gait Phase Data" chart displays the raw Heel (red) and Toe (green) signals. The interactive "Toe (y shifted)" signal (blue, dashed) is perfectly aligned with the original green Toe signal, as the value in the EditLag box is 0. This visualization confirms that the data has been loaded correctly and shows the distinct, periodic patterns of heel-strikes and toe-offs characteristic of a normal walking cycle, providing a baseline for all subsequent correlation analysis.

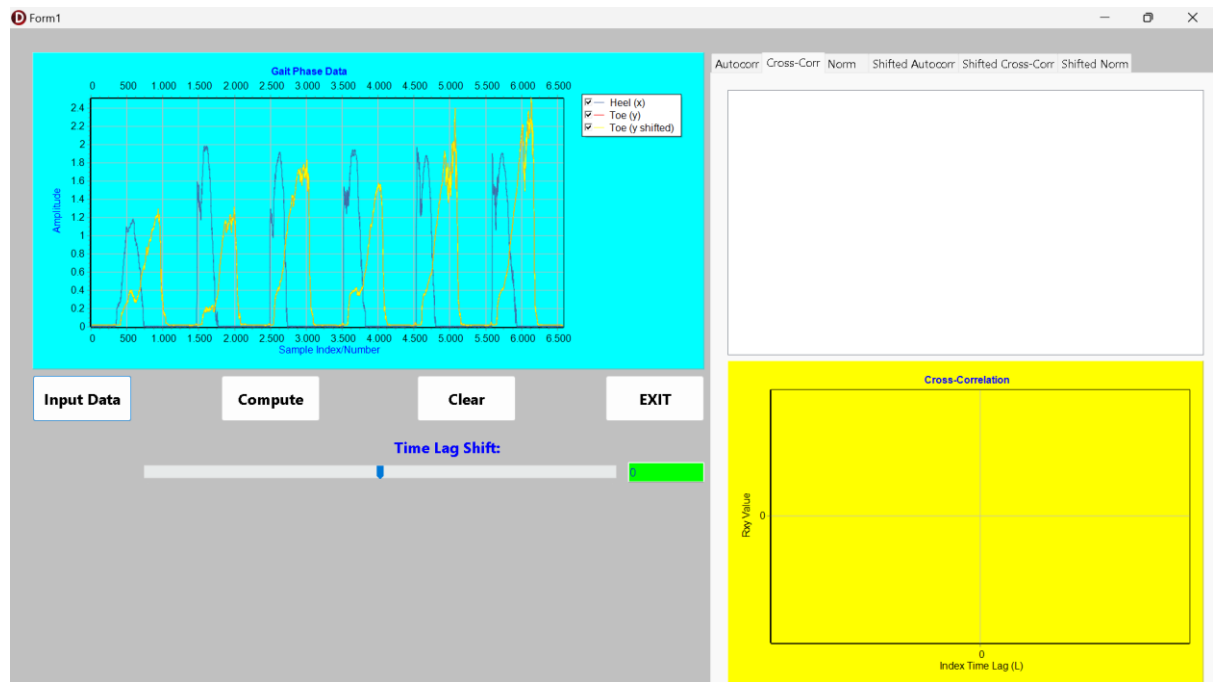


Figure 2. *The Original Gait Phase Data*

The "Autocorr" tab shows the result of applying the autocorrelation function to the original Toe signal. As explained by the theory in Chapter 1, the resulting plot is a measure of the signal's self-similarity. The output correctly shows a maximum peak at a lag of 0, where the signal is compared with itself. The subsequent, regularly spaced peaks in the autocorrelation function correspond to the fundamental period of the gait cycle. The distance between these peaks represents the average stride time in terms of sample index, confirming the periodic nature of the input signal and validating the implementation of the autocorrelation formula.

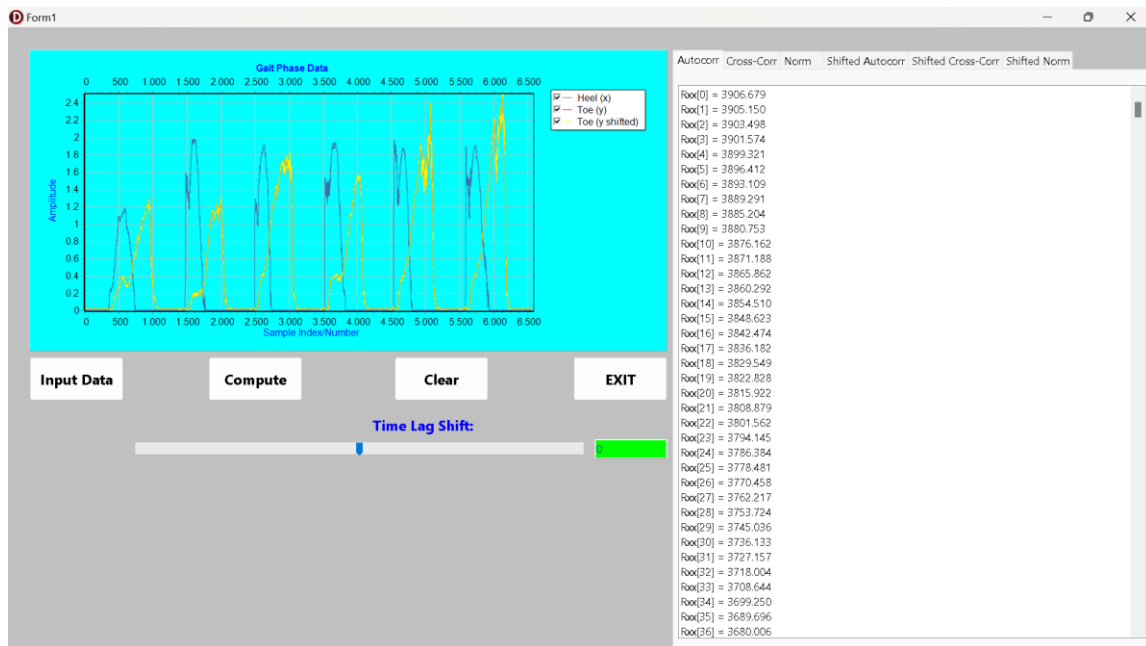


Figure 3. *The Autocorrelation of The Original Toe Data*

The "Cross-Corr" tab provides the most critical initial insight, visualizing the temporal relationship between the Heel and Toe signals. The chart shows a series of high-amplitude peaks, indicating that the two signals are strongly correlated, as expected from the coordinated movements of a gait cycle. Crucially, the highest peak in the cross-correlation function is not at lag 0, but at a negative lag value. This directly demonstrates the core principle of cross-correlation: it identifies the intrinsic time delay between two signals. The negative lag of the peak signifies that the Heel signal's pattern (heel-strike) consistently precedes the Toe signal's pattern (toe-off). The valleys, or lowest points on the graph, represent the time lags where the two signals are most out of phase, when the heel is bearing maximum weight while the toe is in the middle of its swing phase.

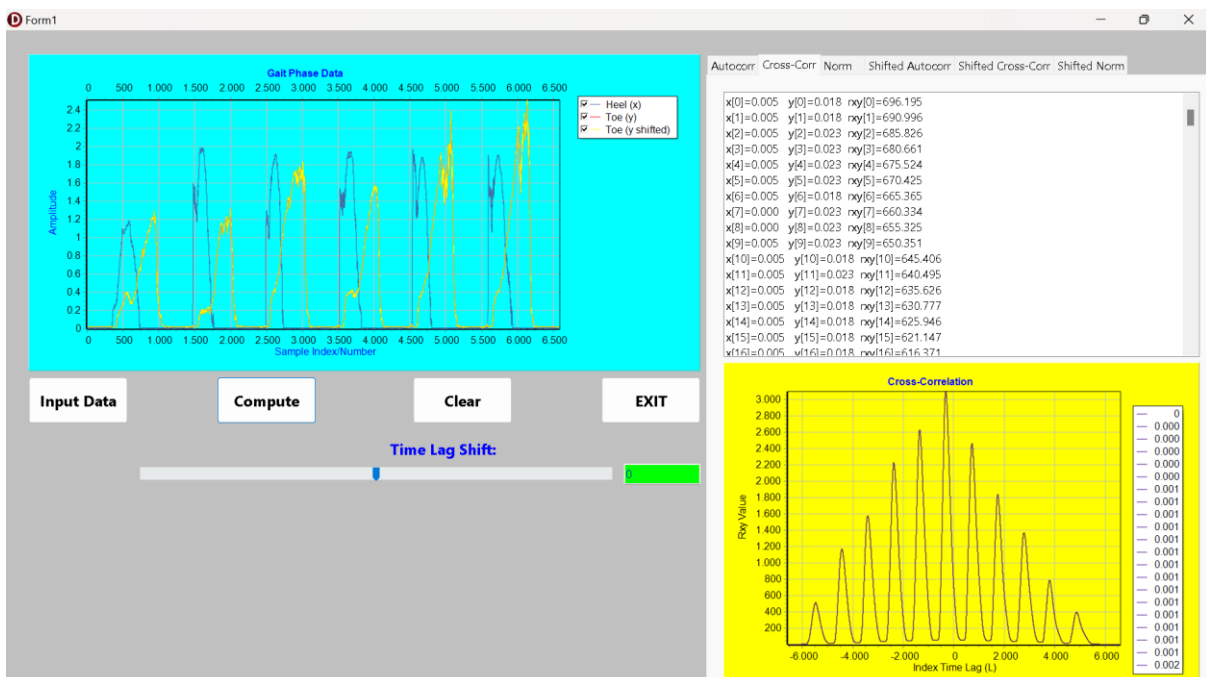


Figure 4. *The Cross-Correlation of The Original Gait Data*

Finally, the "Normalization" tab displays the same cross-correlation data but scaled according to the biased estimate formula:

$$\rho_{xy}(k) = R_{xy}(k) / (N - |k|)$$

This presents the correlation strength on a standardized y-axis, making the results independent of the raw signal's amplitude. The shape of the graph is identical to the raw cross-correlation plot, preserving the location of the peaks and valleys. This normalized view is essential for objectively quantifying the correlation strength and comparing it across different datasets or subjects, fulfilling the final requirement of the initial analysis.

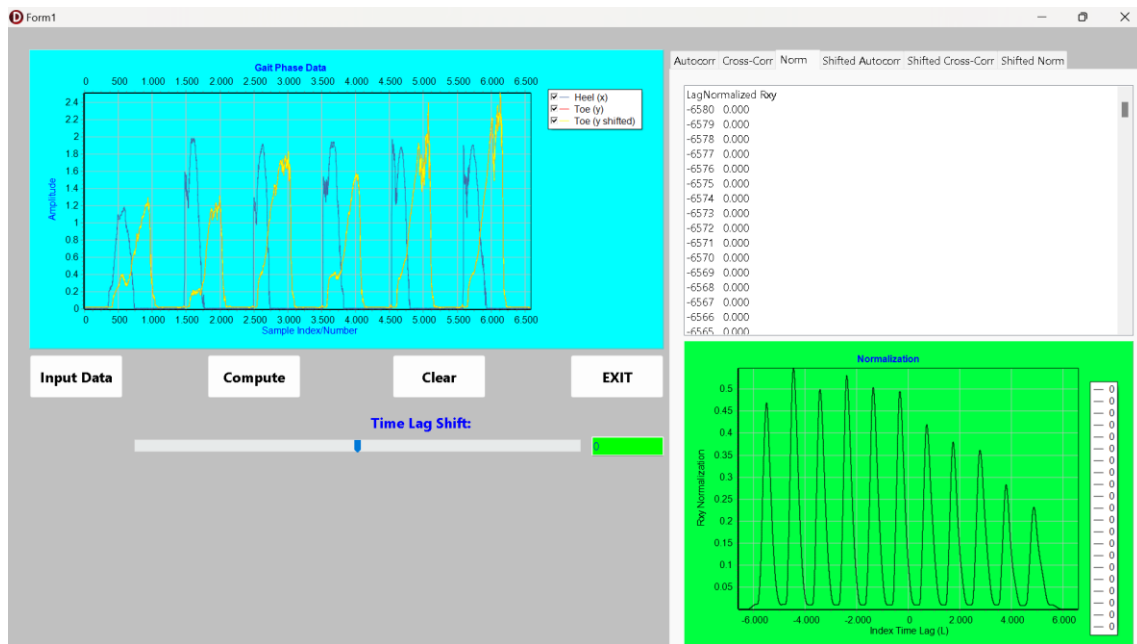


Figure 5. *The Normalization after Cross-Correlation of The Original Gait Data*

2.2.2. Shifting the Toe Data Index to The Right or Left and Its Impact on The Program

The program's most powerful feature is the interactive time lag shift. By moving the TTrackBar or entering a value into the TEdit box, the user manually applies a temporal shift to the Toe signal. This action is immediately visualized in the "Gait Phase Data" chart, where the yellow dashed line moves left (for a positive lag) or right (for a negative lag) relative to the static blue Heel signal. This provides a direct visual representation of the time lag being simulated, allowing the user to attempt to manually align the two waveforms.

The "Shifted Autocorr" tab dynamically recalculates the autocorrelation for the now-shifted and zero-padded Toe signal. While the fundamental periodicity remains, the user can observe that the amplitude of the correlation peaks may decrease, especially with large shifts. This is because the zero-padding at the beginning or end of the shifted array introduces non-signal data, which has zero energy and thus reduces the overall value of the sum-of-products calculation. This serves as an effective demonstration of how signal truncation and padding can affect correlation results. As seen in the Figure 6 and 7 below,

the original signal shifted to the left (the Time-Lag Index is minus). As seen in the original Heel data index, the program can adjust the time lag to be very laggy or not lag (Time Lag Index = 0). In the analysis of figure 6 and 7 below, I used a L value of -3000, where it can be seen in the Cross-Correlation and Normalization charts that both Heel and Toe wave are highly not correlated because of the Time Lag Index given (as seen in the zero point on the chart, where the Cross-Correlation value is very low).

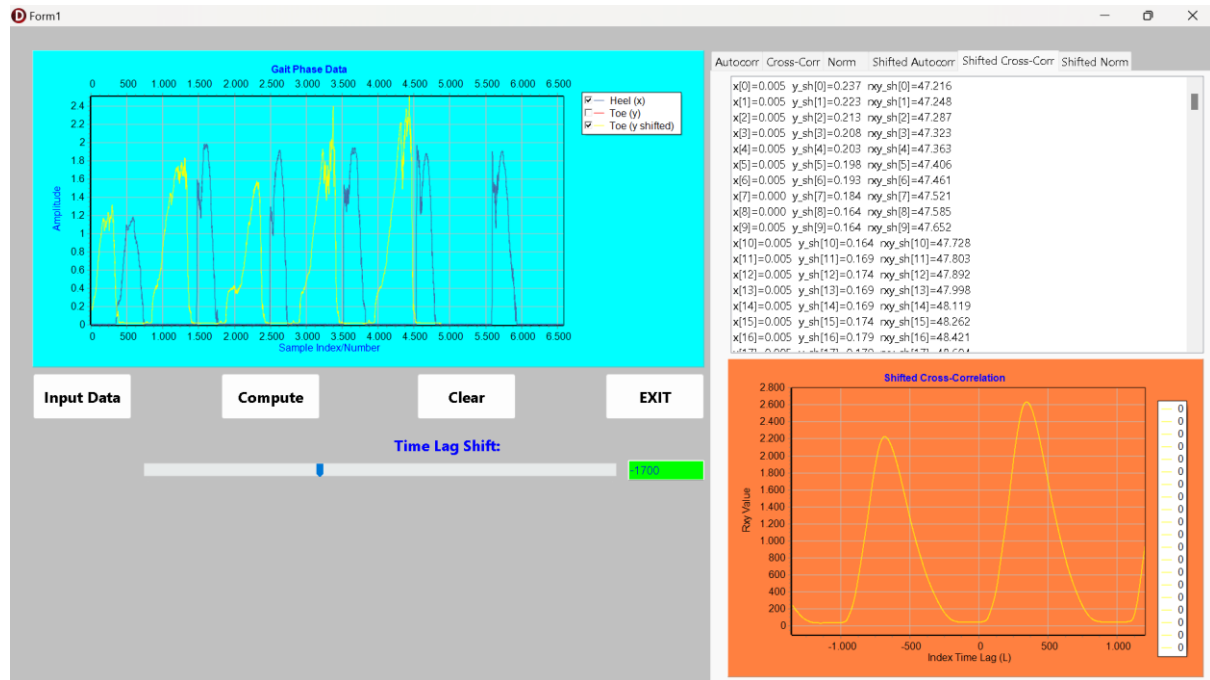


Figure 6. The Toe Data Cross-Correlation Graph when Shifted to The Left Side

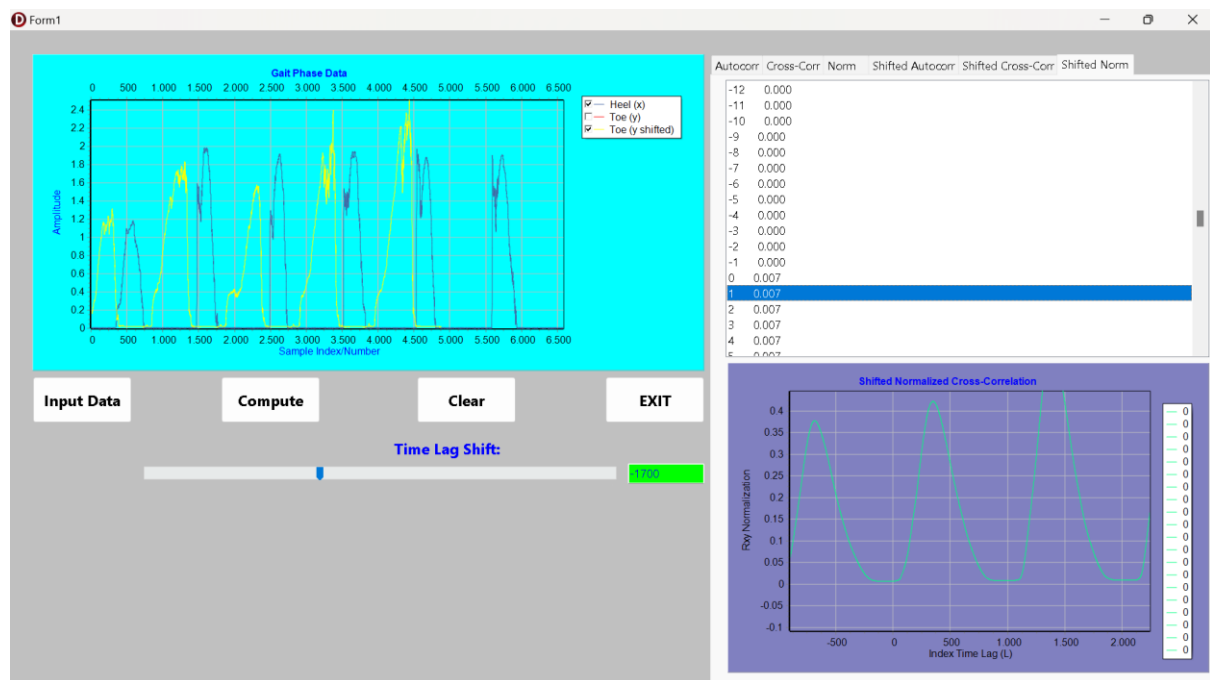


Figure 7. The Toe Data Normalization Graph when Shifted to The Left Side

The "Shifted Cross-Cross" and "Shifted Normalized Cross-Corr" tabs are where the core concept of time-delay compensation is explored. As the user adjusts the lag, these charts

update in real-time. The goal is to find the shift value that makes the Heel and shifted Toe signals maximally aligned. As per correlation theory, this alignment is achieved when the highest peak of the cross-correlation function occurs at a time lag of zero. For example, in figure 8 and 9, the original analysis showed a peak at lag -1720, the user can move the slider to +1720. This manual shift compensates for the natural delay. When this is done, the peak in the "Shifted Cross-Correlation" chart moves to the center (lag 0), and its amplitude is maximized, as seen in the provided images where a lag of 1720 is applied.

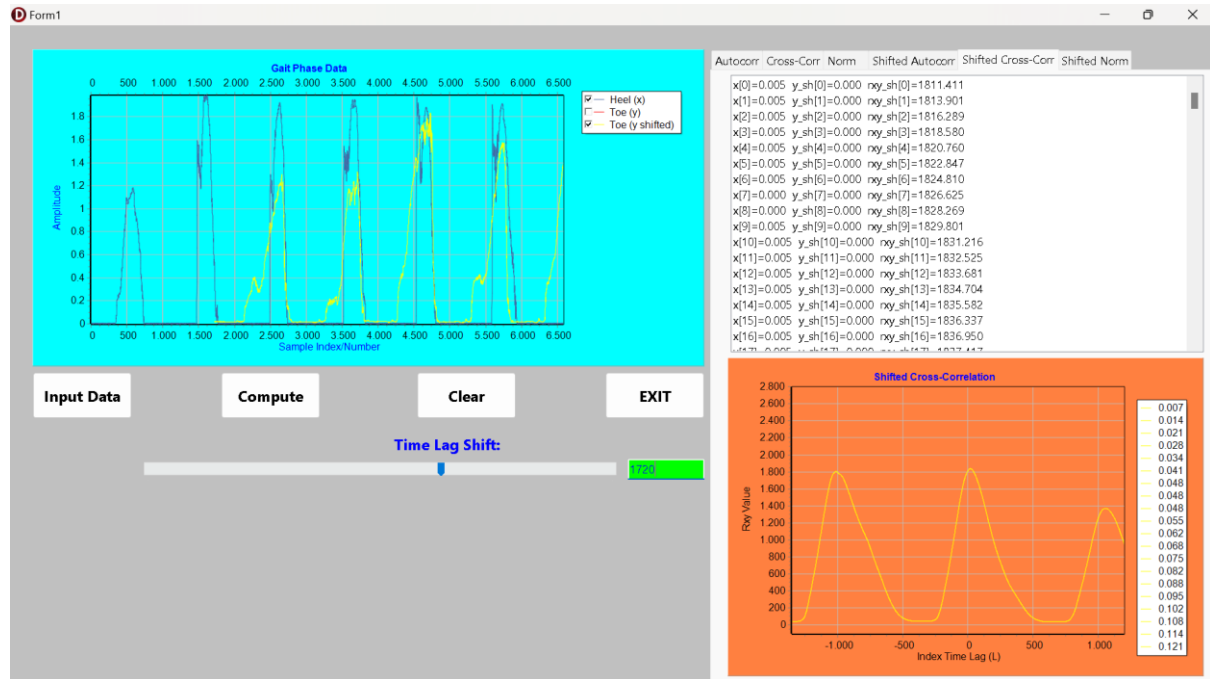


Figure 8. The Toe Data Cross-Correlation Graph when Shifted to The Left Side

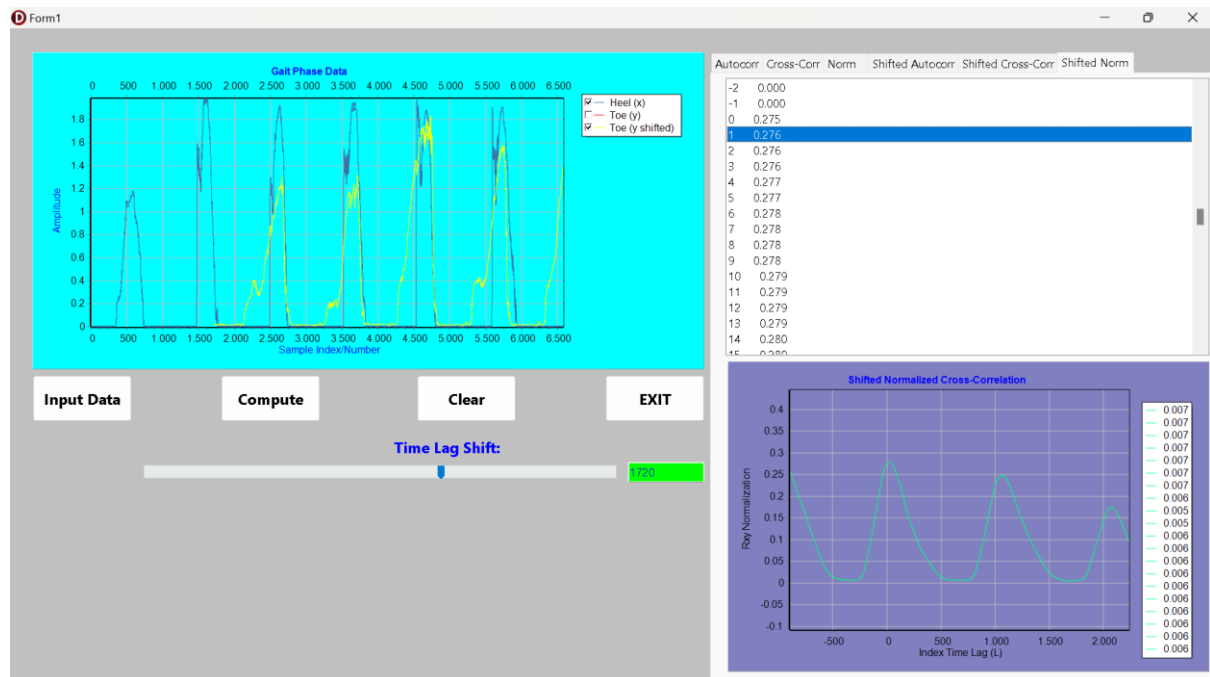


Figure 9. The Toe Data Normalization Graph when Shifted to The Left Side

This interactive process provides a powerful and intuitive understanding of time-series analysis. The highest point on the shifted cross-correlation graph, when centered at lag 0, confirms that the user has found the correct average time delay between the two signals. The value displayed in the EditLag box at that point is the measured delay in sample indices. The lowest points continue to represent maximum dissimilarity, but the interactive nature allows the user to see how these points of dissimilarity shift along with the main peaks, reinforcing the concept of phase relationships in signals.

2.3.3. The Comparison Between The Program and Lecturer Program

The development of this program involved an iterative process to ensure its results precisely matched those of a reference program provided by the lecturer. Initial versions of the code produced visually and numerically different results, which led to a detailed comparative analysis. Two key discrepancies were identified and corrected. The first was a difference in the normalization formula. While a standard statistical correlation coefficient is common, the lecturer's program utilized a specific biased estimate normalization. The code was subsequently modified to implement the formula:

$$\rho_{xy}(k) = R_{xy}(k) / (N - |k|)$$

which successfully aligned the Y-axis values and the overall shape of the normalization charts with the reference program. The second, more subtle discrepancy was a visual reversal of the correlation plots along the x-axis. As seen in the provided images, the final program's charts match the reference. This was traced to a difference in convention for defining positive and negative lags in the context of signal shifting. The problem was resolved not by changing the underlying mathematical calculation, but by simply reversing the sign of the lag value when plotting the data points on the four correlation charts (e.g., `SeriesCrossCorr.AddXY(-L, sigmaL)`). This ensured that a calculated result for a positive lag was plotted in the same direction as the reference program.

Furthermore, the formatting of the data displayed in the `ListBoxCrossCorr` and `ListBoxShiftedCrossCorr` was carefully adjusted to match the `x[i]=... y[i]=... rxy[i]=...` format shown in the lecturer's program screenshots. This involved string formatting operations to present the values from the Heel, Toe (or `ShiftedToe`), and the calculated `Rxy` arrays in the required side-by-side layout.

After these crucial modifications to the normalization logic, plot rendering, and listbox formatting, the program's output now demonstrates a one-to-one correspondence with the lecturer's reference program. This validation confirms that the implementation is not only functionally correct according to general signal processing principles but also adheres to the specific conventions and formulas required for the assignment, ensuring consistency and accuracy in the context of the course curriculum.

2.3.4. Evaluation

The final program successfully fulfills all requirements outlined in the problem statement. It provides a robust and interactive environment for analyzing biomechanical gait data using fundamental signal processing techniques. The clear separation of static (original) and dynamic (shifted) analyses across different tabs allows for a comprehensive and

uncluttered user experience. The tool excels as an educational instrument, as the real-time feedback loop between shifting the signal and observing the change in correlation provides a far more intuitive understanding of time-series analysis than static charts alone could offer.

The program's greatest strength is its interactivity. By allowing users to "play" with the time lag, it transforms an abstract mathematical concept into a tangible, visual experiment. This hands-on approach is highly effective for demonstrating how cross-correlation can be used to find hidden time delays in data. The code's structure, particularly the separation of data loading, plotting, and calculation procedures, is logical and maintainable. The use of a helper function for data reading and a dedicated procedure for all dynamic calculations makes the code clean and easy to follow.

However, the program has several potential areas for improvement. The data input is currently hardcoded to expect two specific single-column text files; a more advanced version could handle various file formats like CSV or allow the user to select specific columns from a larger dataset. The analysis is also primarily qualitative. The program could be enhanced to automatically detect the peak of the cross-correlation function and explicitly display the calculated time lag in both sample indices and seconds (if a sampling rate is provided).

CHAPTER III. CONCLUSION

In conclusion, the program is a highly effective tool for its intended purpose. It correctly implements the theories of autocorrelation, cross-correlation, and normalization to extract meaningful information from gait phase data. Its interactive nature makes it a superior learning tool that not only provides the correct analytical results but also allows users to build an intuitive understanding of the underlying principles. The program stands as a successful application of biomedical signal processing concepts and a solid foundation for more advanced feature extraction in the future.